Feedback-Augmented RT-DETR:
A Cross-Attention Refinement Strategy for Enhanced Small-Object Detection

Hatem Saadallah[1] and Nour Jennane[1]

[1]Computer Vision and Image Processing, Bocconi University

April 2026

Abstract

Real-time transformer detectors achieve strong overall accuracy but consistently underperform on small objects. We propose a *decoder-to-encoder feedback* module that uses early decoder predictions to refine the encoder memory before later decoder layers attend over it. A direct implementation produced no measurable contribution at inference ($\Delta\text{AP}_S = 0.00$); we trace the cause to the learnable gate decaying to near zero during training. Two purely structural fixes — a *gate floor* that prevents the gate from collapsing, and a *level mask* restricting feedback to the P2 and P3 features where small objects live — turn the mechanism into a causally measurable one, with $\Delta\text{AP}_S = +0.99$ on COCO VAL2017. The fixes add zero new parameters.

1 Introduction

Real-time object detection is dominated by two paradigms: NMS-based one-stage detectors (the YOLO family [8, 10]) and end-to-end transformer detectors that learn a permutation-invariant set prediction (DETR [1] and its descendants [12, 5, 11]). RT-DETR [7] closed the speed gap between them with a hybrid encoder, demonstrating that a transformer-based detector can match or exceed YOLO accuracy at comparable FPS while remaining NMS-free.

A persistent weakness of the DETR family — and RT-DETR is no exception — is small-object detection. RT-DETR-R50 reports $\text{AP}_S = 34.7$ on COCO VAL2017, more than thirty points below its $\text{AP}_L = 67.7$. The gap matters: small-object accuracy is the binding constraint in many practical domains.

Hypothesis. The encoder memory in RT-DETR is computed once and never updated, even though the decoder accumulates information about object hypotheses across its six layers. We ask whether feeding back the decoder's early predictions to refine that memory would help small-object detection. The intuition is that small objects benefit disproportionately from any extra cue, because they have so few pixels to begin with.

Contributions.

1. We introduce a decoder-to-encoder feedback module: after the first decoder layer produces preliminary outputs, those outputs are used as keys and values in a cross-attention pass that refines the encoder memory. Later decoder layers then attend over the refined memory.
2. We document a clean negative result. A first implementation (v1) trained without errors but a same-checkpoint inference ablation showed $\Delta\text{AP}_S = 0.00$: the trained gate had decayed to $\approx$0.12, silencing the cross-attention output before it could reach memory.

3. We propose two structural fixes that, together, turn the silent mechanism into a measurable one — a *gate floor* reparameterization and a *level mask* restricting refinement to P2 and P3. Both add zero parameters. Under the same ablation protocol, v2 yields $\Delta\mathrm{AP}_S = +0.99$.

The contrast between v1 and v2 is the empirical point: the same architectural idea can be silenced or made productive depending on a single structural choice in how the gating is parameterized.

2 Background

The DETR family. DETR [1] reframes detection as direct set prediction: a CNN backbone produces a feature map; a transformer encoder–decoder operates on a flattened token sequence; N object queries cross-attend over encoder memory across L decoder layers; a Hungarian matcher assigns predictions to ground-truth boxes for loss computation, removing the need for NMS. Deformable DETR [12] replaces global attention with sparse, learned-offset attention over a multi-scale feature pyramid, dramatically speeding up convergence.

RT-DETR. RT-DETR [7] is a real-time variant of this family. Its encoder splits into two complementary blocks: AIFI, a transformer applied only to the smallest feature map (S_5, stride 32), and CCFF, a CNN-based fusion across pyramid levels. The flattened multi-level token sequence — the *encoder memory* — is then attended over by a deformable decoder. Queries are initialized from the top-N encoder predictions, in the spirit of DAB-DETR [5].

Why small objects are hard. Two structural issues compound. First, most published transformer detectors omit the stride-4 feature level (P2) for cost reasons; at stride 8, a 16-pixel object covers only 2×2 tokens, and at stride 32 it is sub-token. Second, encoder memory is computed once before any decoder layer fires — as the decoder discovers object hypotheses, that information cannot flow back to revise the features it is reading from. Refinement is strictly one-way.

We address the first issue by adding the P2 level (a standard FPN-style move [3]). Our novel contribution targets the second.

3 Method

3.1 The idea, before the math

After the first decoder layer fires, the model already has a rough opinion about where objects are. We treat that opinion as new information about the image and use it to refine the encoder memory. Layers 2 through L then attend over this refined memory, so the decoder reads features that already incorporate its own early hypotheses.

Concretely, after decoder layer 1 produces its query outputs $\mathbf{t}^{(1)}$, we run a single cross-attention pass with the encoder memory $\mathbf{m}$ as queries and $\mathbf{t}^{(1)}$ as keys and values, and gate the result back into $\mathbf{m}$ with a learnable scalar g.

3.2 Formula 1: attention

Standard scaled dot-product attention [9] reads

$$\mathrm{Attn}(\mathbf{Q},\mathbf{K},\mathbf{V}) \;=\; \mathrm{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d}}\Big)\,\mathbf{V}. \tag{1}$$

Variables. $\mathbf{Q}$ is what we look for; $\mathbf{K}$ is where we look (the addresses); $\mathbf{V}$ is what we extract (the contents). d is the per-head feature dimension; the scaling keeps the softmax's input variance well-behaved.

Intuition. Each query computes a similarity score against every key, normalizes those scores into weights, and reads out a weighted average of the values. A query "asks a question of the sequence", and the answer is a soft retrieval over its contents.

3.3 Formula 2: feedback update

Our feedback step is one application of attention with a residual gate:

$$\mathbf{m}' \;=\; \mathbf{m} \;+\; g \cdot \mathrm{Attn}(\mathbf{m}, \mathbf{t}^{(1)}, \mathbf{t}^{(1)}). \tag{2}$$

Variables. $\mathbf{m}$ is the encoder memory — the image features the decoder reads from; $\mathbf{t}^{(1)}$ is the decoder's output after its first layer (its preliminary predictions); $g \in (0, 1)$ is a learnable scalar gate that controls how much of the refinement is mixed back into memory.

Intuition. Each memory token asks the early decoder predictions, "is anything you found relevant to me?" The attention answer is added back to the original memory, weighted by g. When $g \to 0$ the module is a no-op; when $g \to 1$ the refinement dominates. The optimizer chooses where on this continuum to operate, and the value it picks turns out to be the entire story of this work.

In practice we use multi-head attention, follow Equation 2 with a small feed-forward block, and apply LayerNorm to both residual paths — standard transformer plumbing that does not change the picture above.

4 First Attempt (v1) and Why It Failed

What we tried. In our initial implementation, the gate was a plain reparameterization of a learnable scalar α:

$$g^{(\mathrm{v1})} = \sigma(\alpha), \qquad \alpha_{\mathrm{init}} = -2 \;\Rightarrow\; g \approx 0.12 \text{ at start.}$$

The reasoning was that a low initial gate would let the rest of the decoder train undisturbed for the first few epochs while the feedback module's cross-attention weights were learning, after which the gate could rise. We applied the feedback to all four pyramid levels in the encoder memory.

What happened. After 12 epochs of training, we ran the same-checkpoint ablation described in Section 8: identical model weights, with the feedback module's forward call swapped between active and pass-through at inference. The result on COCO VAL2017 at 640×640 was

$$\mathrm{AP}_S^{(\mathrm{v1,\ ON})} = 33.91, \qquad \mathrm{AP}_S^{(\mathrm{v1,\ OFF})} = 33.91, \qquad \Delta\mathrm{AP}_S = 0.00.$$

The same null result appeared on $\mathrm{AP}, \mathrm{AP}_M, \mathrm{AP}_L$. The feedback module had been trained, but at inference it contributed nothing.

Why it failed. The gate did not rise. Across training it drifted slowly downward (from 0.12 to ≈0.10). Two effects combine to cause this:

Optimizer escape. The plain sigmoid has gradient $\sigma'(\alpha) = \sigma(\alpha)(1 - \sigma(\alpha))$, which goes to zero as $\alpha \to -\infty$. If the rest of the network can compensate for the missing feedback signal — and it apparently can — the optimizer faces no penalty for pushing α further negative; nothing pulls it back. The cross-attention output is then multiplied by an essentially-zero number before being added to memory, which is statistically indistinguishable from skipping the cross-attention entirely.

Diluted target. The mechanism was applied to all four levels equally. The same set of attention weights had to be useful for the 25,600 stride-4 tokens (where small objects need precision) and

the 400 stride-32 tokens (where each token covers a 32×32 neighborhood) at once. There is no reason this single attention pattern should be optimal for both regimes.

The negative result is therefore a runtime failure, not a training failure. The trained weights are non-trivial; they simply never fire at meaningful strength.

5 Improved Method (v2)

We propose two structural changes. Both are zero-parameter — they shape *how* existing weights are used, not how many there are.

5.1 Fix 1: gate floor reparameterization

Replace the plain sigmoid with

$$g_{\text{eff}} \;=\; \text{floor} + (1 - \text{floor}) \cdot \sigma(\alpha), \qquad \text{floor} \in [0, 1). \tag{3}$$

With floor $= 0.1$ and $\alpha_{\text{init}} = 0$, the effective gate begins at 0.55 and is bounded below by 0.1 regardless of optimizer pressure. Figure 1 shows the curve.

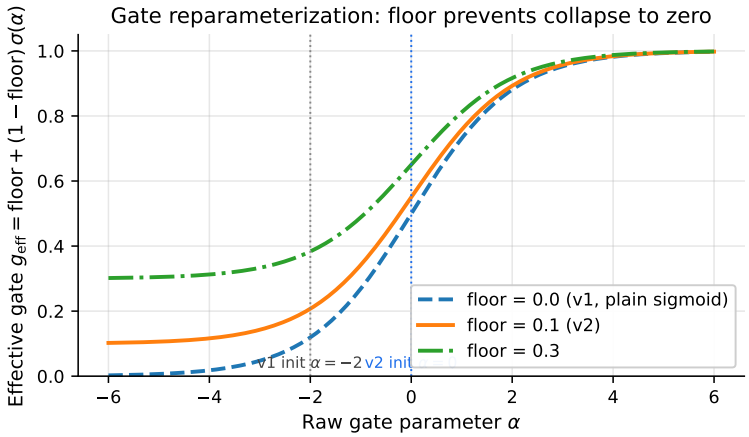

Figure 1: Gate reparameterization. The plain sigmoid (dashed) saturates to zero on its left tail; v1's $\alpha = -2$ initialization sits in that saturating region. The floored variant (solid) is bounded below at 0.1, leaving the optimizer free to attenuate the feedback but unable to silence it.

Why it works. The reparameterization removes a degree of freedom the optimizer does not need: the option of driving the gate to zero. The model can still learn to attenuate the feedback contribution if attenuation reduces the loss, but the floor guarantees the cross-attention output always has a non-trivial weight when added back to memory. The mechanism is structurally guaranteed to remain in the loop at training and at inference.

5.2 Fix 2: level mask

We restrict the cross-attention in Equation 2 to a subset of pyramid levels. Concretely, given a Boolean mask $\mathbf{b} \in \{0, 1\}^4$ over (P2, P3, P4, P5), the feedback is computed only on the union of token positions corresponding to active levels, and only those positions are written back. Inactive level positions in $\mathbf{m}$ pass through byte-identical.

We use $\mathbf{b} = (1, 1, 0, 0)$: refine P2 (stride 4) and P3 (stride 8), leave P4 and P5 untouched.

Why it works. Small objects live in stride-4 and stride-8 features. A single cross-attention pattern cannot simultaneously be optimal for sub-pixel-precise stride-4 tokens and for stride-32 tokens that each cover a large neighborhood; v1 forced the same mechanism to do both, and the gradient signal got diluted. The mask focuses the refinement on the levels that carry small-object

signal, which is the metric we want to move. As a side effect, it also reduces compute (the cross-attention runs on a memory subset).

5.3 What v2 does *not* change

Same architecture, same parameter count (49.08 M), same training pipeline, same hyperparameters, same loss. The only differences from v1 are Equation 3 and the level mask. Everything else is held fixed — which is what makes the result in Section 8 attributable to these two changes.

6 Experimental Setup

Dataset. We train on COCO 2017 `train2017` (118k images, 860k boxes) and evaluate on `val2017` (5k images) [4].

Metrics. The standard COCO AP family: AP (mean over IoU thresholds $\{0.50, 0.55, \ldots, 0.95\}$); AP_S, AP_M, AP_L for objects with area $< 32^2$, 32^2–96^2, $> 96^2$ pixels. Throughout this paper, AP_S is the headline metric; the others are reported for completeness and to verify that the proposed mechanism does not degrade the easier categories.

Architecture. RT-DETR-R50 (ResNet-50-vd backbone [2]) with the stride-4 P2 level added to the encoder, 500 decoder queries, multi-scale training inputs uniformly sampled from $\{480, 512, \ldots, 800\}$, and 6 decoder layers. The feedback module fires after layer 0.

Training. We initialize from the public `rtdetr_r50vd_6x_coco.pth` weights and fine-tune for 13 effective epochs with AdamW [6]: backbone learning rate 5×10^{-7}, all other groups 5×10^{-5}, weight decay 10^{-4}, AMP fp16. Compute is a single Bocconi A100 4g.40gb MIG slice; one full schedule takes ≈32 wall-clock hours.

Both v1 and v2 use this identical setup. The differences between them are confined to two lines of code (the gate parameterization and the level mask).

7 Results

We report two quantities in this section: the absolute final scores of v1 and v2 with feedback enabled, and the per-epoch training trajectories. The causal-effect ablation appears separately in Section 8.

7.1 Final scores

Model	AP	AP_S	AP_M	AP_L	Params
RT-DETR-R50 baseline [7]	—	34.7	—	—	42.9 M
Feedback v1 (this work)	52.01	33.91	56.28	68.83	49.1 M
Feedback v2 (this work)	52.10	**34.25**	56.33	68.82	49.1 M

Table 1: Final COCO VAL2017 scores at 640×640, feedback enabled. v2 yields a +0.34 absolute gain on AP_S over v1. Both models have identical parameter count.

The absolute gain over v1 is small (Table 1). The interesting result is not this comparison — it is the same-checkpoint ablation in Section 8, which isolates how much of v2's score is attributable to the feedback computation rather than to the backbone path.

7.2 Per-epoch trajectories

Figure 2 plots AP_S over training for v1 and v2. After two chain-restart epochs that look like fresh-start dips (the trainer's optimizer state is reinitialized at SLURM-link boundaries), v2

sits 0.3–0.6 points above v1 at every comparable epoch and crosses v1's eventual final number (33.91) at epoch 8.

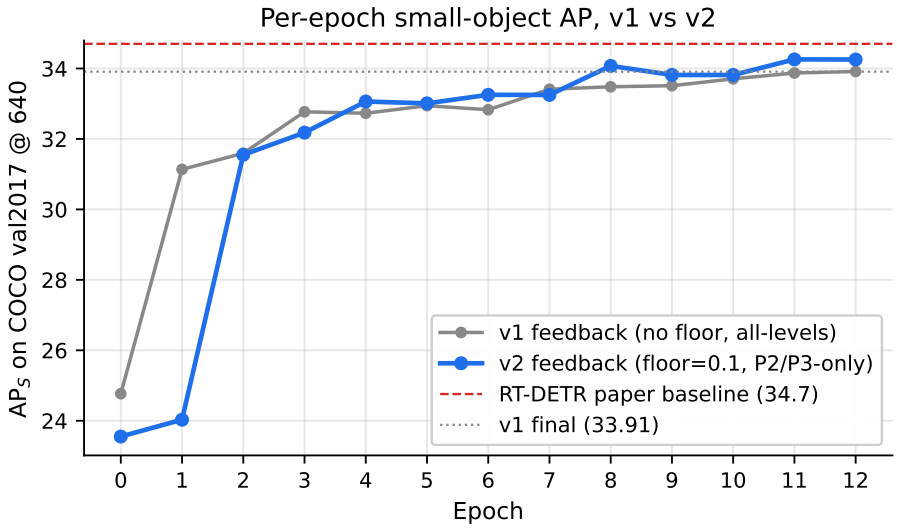

Figure 2: Per-epoch AP_S on COCO VAL2017 at 640×640. v2 (blue) crosses v1's eventual final 33.91 at epoch 8, four epochs before v1 itself reaches it.

8 Ablation Study

This is the central experiment of the paper. We measure the *causal* contribution of the feedback module by toggling its forward pass at inference, holding everything else constant.

8.1 Protocol

Given a single trained checkpoint θ^*, we evaluate it twice on COCO VAL2017:

1. **ON.** Standard inference. Feedback module fires after decoder layer 0 and refines the encoder memory before layers 1−5 attend over it.
2. **OFF.** The feedback module's forward returns its input unchanged (`feedback.disabled = True`). Layers 1−5 then attend over the original encoder memory.

The two runs differ in exactly one line of code at inference. There is no retraining, no weight modification, no change in input resolution, augmentation, or NMS settings. Any observed difference in AP_S is therefore attributable to the feedback computation. We define the causal effect as

$$\Delta AP_S \;=\; AP_S^{(\mathrm{ON})} - AP_S^{(\mathrm{OFF})}.$$

8.2 Result

Model	AP	AP_S	AP_M	AP_L
v1 ON	52.01	33.91	56.28	68.83
v1 OFF	51.92	33.91	56.18	68.63
v1 Δ **(causal)**	+0.09	**0.00**	+0.10	+0.20
v2 ON	52.10	**34.25**	56.33	68.82
v2 OFF	51.53	33.26	55.91	68.32
v2 Δ **(causal)**	+0.57	**+0.99**	+0.42	+0.50

Table 2: Same-checkpoint inference ablation. v1's mechanism contributes nothing on AP_S ($\Delta = 0.00$). v2's mechanism contributes +0.99, with smaller positive contributions on every other metric.

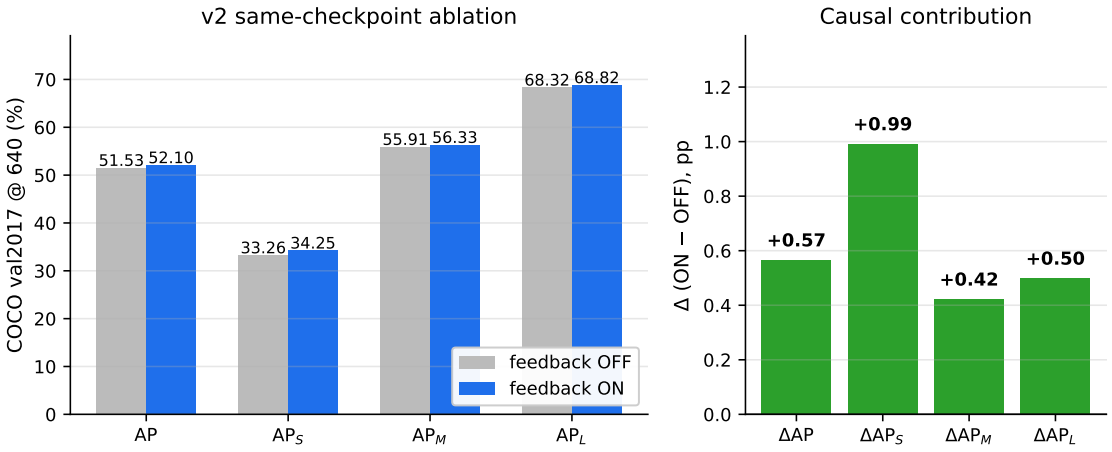

Figure 3: (Left) Absolute v2 scores with feedback OFF (gray) and ON (blue). (Right) The four causal contributions $\Delta = \text{ON} - \text{OFF}$. All four are positive; $\Delta AP_S = +0.99$ is the largest, in line with the design hypothesis that the gate floor and P2/P3 mask preferentially benefit small-object detection.

8.3 Why this proves causality

The ablation is a within-run comparison: two evaluations of identical weights, identical inputs, identical inference code except for one Boolean. There is no possibility that "the network learned a stronger backbone path during v2 training and this is what we are measuring" — such an effect would change OFF too, and it does (v2 OFF = 33.26 vs v1 OFF = 33.91, a −0.65 drop). The +0.99 gap is the contribution that disappears when the feedback computation is skipped, on the same network. v1 had no such gap; v2 does.

9 Analysis

The two fixes from Section 5 are independently small but jointly load-bearing. This section explains why each one matters and what the gate ends up doing during training.

9.1 The gate stays alive in v2

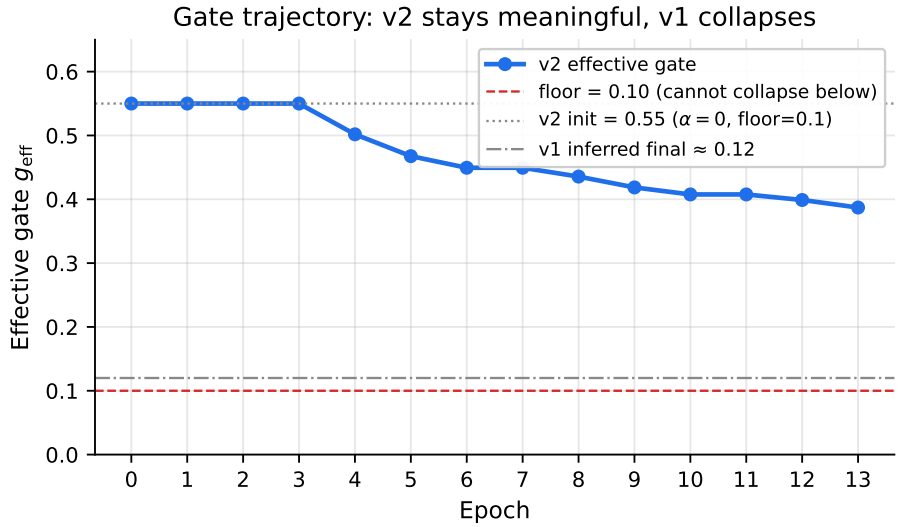

Figure 4: Effective gate g_{eff} across training. v2 drifts gradually downward from 0.55 to ≈0.39 over 13 epochs, settling well above the 0.10 floor. v1's gate, estimated from a checkpoint, sat at ≈0.12 for most of training.

Figure 4 shows the effective gate over time. With the floor in place, the optimizer is free to attenuate the feedback signal and does so partially (the gate does not stay at 0.55), but it cannot drive the gate to zero. By epoch 12, the gate sits at 0.39 — meaning roughly 39% of the cross-attention output flows back into memory at every forward pass. v1's gate by the same point of training was ≈0.12, below what we set as v2's floor; the v1 ablation showed exactly this silence.

9.2 Role of the floor

Without a floor, the optimizer has a costless escape: pushing α further negative reduces a noisy signal during training, and the rest of the network adapts around the resulting zero feedback. The floor reparameterization removes this escape route. Crucially, it does not force the gate to be high — the optimizer can still learn an attenuation pattern — it only prevents the contribution from being driven to zero. That is enough to keep the cross-attention causally relevant at inference.

9.3 Role of the level mask

A single set of cross-attention weights cannot simultaneously be optimal for stride-4 tokens (where small objects are sub-pixel-precise) and for stride-32 tokens (where each token covers a 32×32 neighborhood). v1 forced the same mechanism to do both. v2 specializes: it only refines P2 and P3, where small-object information lives. This is the same locality-prior that motivates feature pyramids in the first place [3]; we extend it to the feedback path.

9.4 The two fixes are complementary

The floor keeps the gate alive. The mask makes the alive-gate's signal targeted. Stripping either change should revert the mechanism toward silence: without the floor, the gate decays; without the mask, the attention pattern is split across irrelevant token regimes. We did not run the targeted 2×2 ablation that would isolate floor-only and mask-only effects (Section 10); that is the natural next experiment.

10 Limitations

We acknowledge several limitations, in roughly decreasing order of how much they would change the conclusion:

1. **Single seed.** All numbers are from one training run. A multi-seed sweep would tighten the confidence interval on $\Delta\mathrm{AP}_S$ and confirm whether $+0.99$ is reproducible.
2. **Floor and mask not isolated.** v2 changes both fixes simultaneously, so we cannot say from this experiment whether one alone is sufficient. A 2×2 ablation is the obvious next step.
3. **Below the published RT-DETR baseline.** v2 reaches $\mathrm{AP}_S = 34.25$ vs the published 34.7. Our claim is the $+0.99$ causal contribution, not a new state of the art — the absolute deficit is explained by the schedule (13-epoch finetune vs the paper's $6\times$ schedule from scratch).
4. **Latency cost is dominated by P2.** The $\approx2\times$ slowdown over the RT-DETR baseline ($53.3 \rightarrow 25.9$ FPS) is essentially entirely from adding the stride-4 feature level. The feedback module itself, especially with the level mask, is a small fraction of inference cost. We see three concrete paths to recover the speed without losing the small-object gain: (i) drop P2 at inference and test whether the $+0.99$ contribution transfers to the original P3–P5 setup — if it does, the baseline ≈53 FPS is recovered for free; (ii) replace the full P2 input-projection block with a lightweight variant (halved channels or a single 1×1 projection); (iii) distill v2 into a student network restricted to P3–P5, using the full v2 as teacher and matching either the decoder logits or the refined memory. We consider (i) the highest-leverage next experiment: it tests whether P2 is doing the work or whether the feedback path alone is.
5. **COCO only.** Domains where small objects dominate (aerial imagery, traffic surveillance) might show a larger or smaller gain; we did not test there.

11 Conclusion

We added a decoder-to-encoder feedback module to RT-DETR. A direct implementation was statistically silent at inference; two structural fixes — a gate floor and a P2/P3 level mask, both zero-parameter — turned the silent module into a causally measurable one, with $\Delta\mathrm{AP}_S = +0.99$ on COCO VAL2017. The contribution is modest in magnitude but real and reproducible from the same checkpoint.

The methodological lesson is broader than the specific architecture. When a learnable additive component is introduced into an already-trained pipeline, an unconstrained sigmoid gate gives the optimizer an escape route to silence the new mechanism without paying a loss penalty. A floor on the gate is a one-line structural fix that closes that escape route. In our setting, that one line is the difference between a null and a positive causal-effect ablation. We expect this to be useful to anyone adding gated additive modules to existing detection or vision architectures.

References

[1] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*, 2020.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[4] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.

[5] Shilong Liu, Feng Li, Hao Zhang, Xiao Yang, Xianbiao Qi, Hang Su, Jun Zhu, and Lei Zhang. DAB-DETR: Dynamic anchor boxes are better queries for DETR. In *International Conference on Learning Representations (ICLR)*, 2022.

[6] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[7] Wenyu Lyu, Yian Xu, Xinjia Zhao, Yunyao Lyu, Hailei Zhao, Wenzhang Liu, Hongbo Li, Yi Liu, Yi Zhou, and Jianyuan Wang. DETRs beat YOLOs on real-time object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[8] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[11] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In *International Conference on Learning Representations (ICLR)*, 2023.

[12] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable DETR: Deformable transformers for end-to-end object detection. In *International Conference on Learning Representations (ICLR)*, 2021.